

# Lattice.scala: Algorithmic Guide to Every Method

A method-by-method explanation of purpose, control flow, state changes, caches, returned values, computational cost, and relevant edge cases.

**Core idea.** This class incrementally maintains an approximate FCA-like lattice. CVA models supply learned classification, distance, intent, and equivalence; strict symbolic extent inclusion supplies the graph order. Parent nodes are more general and have strict supersets of their children's symbol sets.

## Contents

---

- |                              |                                     |
|------------------------------|-------------------------------------|
| 1. State and accessors       | 8. Reporting and intersection       |
| 2. Cached learned predicates | 9. Semantic relations               |
| 3. Bounded generator search  | 10. Validation                      |
| 4. Materialization           | 11. Auxiliary maintenance           |
| 5. Recursive insertion       | 12. Construction utilities          |
| 6. Symbolic edge helpers     | 13. Complexity and behavior summary |
| 7. Main addIntent            |                                     |

## 1. Class state and accessors

---

The lattice stores nodes in insertion order in `nodeMap: LinkedHashMap[Long, Node]`. IDs come from an `AtomicLong`. Five fixed bounds limit insertion search width, visits, recursion, and child exploration. These bounds trade global completeness for predictable local cost.

### nodes

```
def nodes: Vector[Node]
```

**Algorithm.** Traverse `nodeMap.values` in linked-map iteration order and copy them into an immutable `Vector`.

**Behavior.** This is a snapshot of node references, not a deep copy. Later lattice mutations do not add or remove entries from the returned vector, but mutations to a referenced `Node` remain visible. Time and extra space are both  $O(N)$ .

### size

```
def size
```

Returns the number of entries currently in `nodeMap`. It is an  $O(1)$  map-size lookup. Scala infers the result type as `Int`.

### node

```
def node(id: Long): Node
```

Looks up an ID with `nodeMap(id)`. Average lookup is  $O(1)$ . It returns the exact mutable node object. If the ID is absent, the map's indexing operation throws `NoSuchElementException`; the method does not return an `Option`.

## 2. Cached node construction and learned predicates

---

The caches in these methods are supplied by `addIntent` and live for one insertion. They are safe because existing accepted nodes are not refitted until structural work ends.

### **targetNode**

```
def targetNode(targetCache, instances): Node
```

1. Extract every instance symbol and form a `Set[String]` key.
2. Look up that key in `targetCache`.
3. If absent, deduplicate instances by symbol, allocate a new ID, fit a CVA model through `createNode`, cache it, and return it.

The target is not necessarily in `nodeMap`; it can remain a temporary candidate. Repeated construction of the same symbolic extent within one insertion therefore performs only one CVA fit.

Important assumption: a symbol uniquely identifies an instance/vector. Two vectors with the same symbol collapse to one cache key and later occurrence.

### **incomingDistance**

```
def incomingDistance(cache, concept, instance): Double
```

1. Use `concept.id` as the cache key.
2. On a miss, calculate the residual distance from the node's CVA model to the incoming vector.
3. Convert NaN to positive infinity.
4. Cache and return the result.

This value is reused in leaf selection, priority ordering, coherence, root choice, and child ordering. Treating NaN as infinity prevents invalid numerical output from appearing artificially close.

### **coherentWithIncoming**

```
def coherentWithIncoming(coherenceCache, distanceCache, singletonTarget, concept, instance): Boolean
```

1. Obtain the cached concept-to-incoming distance.
2. Look up coherence by concept ID.
3. On a miss, require the first distance to be no greater than `classificationThreshold`.
4. For every member of the concept, measure distance using the singleton target's model and require it to be no greater than `trainingRadiusScale`.
5. Cache the conjunction.

The test is deliberately bidirectional in spirit: the concept must be close to the new item, and the singleton model must regard the concept's members as close. Short-circuiting means the member scan is skipped when the first test fails.

**Nuance:** the second comparison uses the raw `trainingRadiusScale` as a distance threshold. Elsewhere that configuration value scales a training residual when a CVA radius is fitted.

### **intentSubset**

```
def intentSubset(subsetCache, left, right): Boolean
```

Cache the directional predicate `coverage(left, right.instances) ≥ equalityCoverage` under `(left.id, right.id)`. In words: “Does the learned classifier of `left` accept a sufficient fraction of `right`’s extent?”

This is semantic containment, not ordinary set containment, and it need not be symmetric or transitive. Its ordered-pair key is therefore necessary. On a cache miss the work is  $O(|\text{right extent}|)$  classifications.

### 3. Bounded generator search

---

#### **maximalConcept**

```
def maximalConcept(bottomFrontier, coherenceCache, subsetCache, distanceCache, target, instance, preferred): Option[Node]
```

**Purpose.** Find a locally most-specific existing concept that both semantically contains the target and is coherent with the incoming instance.

1. Create a priority queue ordered so smaller incoming distance is dequeued first; lower ID provides deterministic tie-breaking.
2. If `preferred` exists, enqueue only it. Otherwise enqueue up to eight nodes from `bottomFrontier`.
3. Maintain a visited-ID set and stop when the queue is empty or 24 distinct nodes have been visited.
4. For each new node, test `intentSubset(concept, target)` and coherence.
5. If compatible, add it to the compatible frontier and do not enqueue its parents. It is the first compatible node on that bottom-up path, so ancestors are less specific.
6. If incompatible, enqueue all existing parent nodes.
7. Return the compatible node with minimum `(size, id)`, or `None`.

The method reads topology and caches but does not mutate the lattice. It is an approximate best-first search because its starts and visits are capped. With the hard limits, graph exploration is bounded; learned predicate cost still depends on inspected extents.

### 4. Materializing a candidate

---

#### **materialize**

```
def materialize(..., target, instance): Node
```

1. Search for a compatible existing concept with `maximalConcept`.
2. Keep it only if `equivalent(existing, target)` succeeds.
3. If no equivalent node is found locally, insert `target` into `nodeMap`.
4. Update `targetCache(target.symbols)` to the actual selected node.
5. Return that canonical node.

The cache rewrite is important: subsequent construction of the same symbolic target returns the reused lattice node rather than the temporary fitted object. Materialization does not itself create edges. Equivalence performs two extent scans, plus optional common-vector checks.

## 5. Recursive local insertion

---

### **addIntentRecursively**

`def addIntentRecursively(..., acceptedIds, target, instance, preferredGenerator, depth): Node`

**Purpose.** Insert or reuse a target concept under a nearby generator, derive local intersection candidates from generator children, and repair affected cover edges.

#### **A. Find a generator**

Call `maximalConcept`, starting at `preferredGenerator` when supplied. If there is no generator, insert the target, add it to the bottom frontier, and return it. If the generator and target are mutually equivalent, return the generator without adding a node.

#### **B. Guard recursive re-entry**

Construct `callKey = (target.symbols, generator.id)`. If it is already in `activeCalls`, recursive processing has reached the same state; stop the cycle by materializing the target directly. Otherwise mark the key active.

#### **C. Examine nearby generator children**

At maximum recursion depth, examine none. Otherwise, resolve generator child IDs, sort by distance to the incoming instance and ID, and keep at most four. For each original child:

1. If the child semantically contains the target, accept the child as a candidate.
2. Otherwise, reject it if it is incoherent with the incoming item.
3. Otherwise construct the deduplicated union of child and target extents.
4. If the union does not increase the child's size, reuse the child.
5. If it does, obtain a cached target node for the union and recursively insert it, preferring the original child as generator and increasing depth.

#### **D. Maintain a non-redundant parent candidate set**

For every accepted candidate, compare it against the current `newParents` snapshot. If the candidate is semantically contained by an existing parent, discard it. If an existing parent is semantically contained by the candidate, remove that parent. Otherwise append the candidate. The result approximates a maximal, non-redundant local frontier.

#### **E. Materialize and rewire**

Materialize or reuse `target` as `newConcept`. For every new parent candidate:

- If the candidate lies symbolically between the generator and new concept, remove the old direct generator-to-candidate edge.
- Orient and add the candidate/new-concept edge according to `locallyMoreGeneral`.

If this call had a preferred original candidate, connect it below the new concept when symbolic ordering permits, removing a now-bypassed generator edge if necessary.

Then test whether any selected parent already lies between generator and new concept. If none does, link generator and new concept directly in the valid orientation.

#### **F. Finish**

Update bottom-frontier membership from whether `newConcept` has children, remove `callKey` from the recursion guard, and return the new/reused concept.

**Exceptional-path note:** `activeCalls.remove(callKey)` is not protected by `finally`. If an exception occurs after insertion into the guard, the caller's temporary guard can retain the key; the entire guard is nevertheless discarded when that `addIntent` call terminates.

## 6. Symbolic-order and edge helpers

---

### `effectiveSymbols`

```
def effectiveSymbols(acceptedIds, concept, instance): Set[String]
```

If the node belongs to the accepted ancestor closure, return its current symbol set plus the incoming symbol; otherwise return its current symbols. This is a non-mutating preview of the post-insertion extent used while edges are arranged before model refitting.

### `locallyMoreGeneral`

```
def locallyMoreGeneral(acceptedIds, parent, child, instance): Boolean
```

1. Calculate both nodes' effective symbol sets.
2. Require the child set to be a subset of the parent set.
3. Require the sets to differ.

Thus the relation is strict symbolic superset: parent is more general. Set comparison is roughly  $O(|parent| + |child|)$ , subject to hash-set behavior.

### `link`

```
def link(bottomFrontier, acceptedIds, parent, child, instance): Unit
```

1. Reject self-links.
2. Reject links that do not satisfy strict effective-symbol ordering.
3. Add the child ID to the parent and the parent ID to the child, preserving bidirectional adjacency.
4. Remove the parent from the bottom frontier because it now has a child.
5. Add the child to the frontier if it has no children.

Linked sets make repeated insertion idempotent. The method checks order but does not independently prove the edge is a Hasse cover; callers remove known intermediate edges locally.

## 7. Main insertion method

---

### `addIntent`

```
def addIntent(instance: Instance): Node
```

**Purpose.** Coordinate one bounded incremental insertion while keeping learned-model observations stable during structural modification.

1. **Empty case:** create, store, and return a singleton node.
2. **Per-call state:** create target, subset, distance, and coherence caches; initialize `acceptedIds`.
3. **Bottom frontier:** collect up to 32 current leaves in map order, sort by incoming distance and ID, and retain the closest 8.
4. **Recursion guard:** create an empty `activeCalls` set.

5. **Initial generator:** search upward for a maximal compatible concept.
6. **Accepted closure:** if a generator exists, gather it and up to 23 reachable ancestors into `acceptedIds`.
7. **Missing-generator repair:** if absent, sample root nodes, keep coherent ones, choose the closest, materialize `root extent ∪ {instance}`, and link it above the root when possible.
8. **Generator retry:** use the original generator or search again after root/intersection creation.
9. **Structural insertion:** call `addIntentRecursively` on the singleton target.
10. **Delayed model update:** for every accepted ID, add the instance if its symbol is new, then refit that node's CVA model.
11. Return the node representing the inserted singleton target.

**Why delayed refitting matters.** Distance, coherence, and subset caches are keyed only by node IDs. Refitting during traversal could make cached values stale. Structural insertion therefore sees one stable old-model snapshot; accepted extents and models change only at the end.

**Validation note:** `validateInstance` exists but `addIntent` does not call it. An invalid dimension will normally be rejected later by CVA fitting or distance validation; an empty symbol is not explicitly rejected on this path.

### **ancestorClosure (nested in addIntent)**

```
def ancestorClosure(start: Node): Set[Long]
```

1. Push the start node into an `ArrayDeque`.
2. Repeatedly pop the last node, creating depth-first-like traversal.
3. If its ID was not seen, add it and append all existing parents.
4. Stop at an empty deque or 24 visited nodes.
5. Return an immutable ID set.

Cycles would not loop forever because IDs are visited once, and the visit cap provides a second bound. The cap also means not every ancestor is guaranteed to receive the new instance.

## **8. Reporting and public intersection**

---

### **printRandomNodeSymbols**

```
def printRandomNodeSymbols(nodeCount = 5, symbolsPerNode = 2, random = Random): Unit
```

1. Filter nodes whose extent size is at least `symbolsPerNode`.
2. Materialize them as an indexed sequence and shuffle with the supplied random generator.
3. Take at most `nodeCount`.
4. Sort the selected sample by descending size and ascending ID.
5. Format each node plus parent and child IDs and print the lines.

This is a diagnostic side effect, not part of insertion. Supplying a seeded `Random` makes selection reproducible. It may allocate and shuffle  $O(N)$  eligible nodes.

### **intersection**

```
def intersection(a: Node, b: Node): Node
```

Concatenate both extents, deduplicate by symbol, and pass the result to `insertOrMergeCandidate`. In this learned formulation, an "intent intersection" is modeled by fitting CVA to the union of object extents.

The method can insert a node or merge instances into an equivalent node, but it does not establish or rebuild graph edges. Calling it alone can therefore change node content without making topology a complete cover relation.

## 9. Equivalence, coverage, and order

---

### **equivalent**

```
def equivalent(a: Node, b: Node): Boolean
```

1. Compute the fraction of `b` instances classified by `a`.
2. Compute the reverse fraction.
3. Require both to meet `equalityCoverage`.
4. If configured, also require each node to classify the other's common vector.

This is learned mutual equivalence rather than equality of symbol sets. It is symmetric because both directions are explicitly tested. Cost is  $O(|a| + |b|)$  classifications, plus two optional common-vector classifications.

### **semanticallyContains**

```
def semanticallyContains(parent: Node, child: Node): Boolean
```

Return whether the parent model classifies at least `equalityCoverage` of the child extent. This is identical in meaning to an uncached `intentSubset(parent, child)`. It is useful for learned ranking/search but intentionally not the sole topology rule.

### **coverage**

```
def coverage(classifier: Node, xs: Iterable[Instance]): Double
```

1. Traverse `xs` once.
2. Count total instances and those classified by the node.
3. Return `accepted / total` as a double.
4. For an empty input, return 1.0.

Empty coverage is vacuously true. The method is  $O(K)$  classifier calls, where  $K$  is the number of input instances, and uses  $O(1)$  extra space.

### **isMoreGeneral**

```
def isMoreGeneral(parent: Node, child: Node): Boolean
```

Return true exactly when `child.symbols` is a proper subset of `parent.symbols`. Unlike `locallyMoreGeneral`, it uses actual current symbols and does not preview pending accepted updates. It is the authoritative structural relation used by validation and full cover rebuilding.

## 10. Structural validation and description

---

### **validate**

```
def validate(): Unit
```

For every node, inspect both directions of every adjacency:

- Each parent ID must exist.

- The parent must list this node as a child.
- The parent must be strictly more general by actual symbolic extent.
- Each child ID must exist.
- The child must list this node as a parent.
- The current node must be strictly more general than that child.

Violations throw `IllegalArgumentException` through `require`, with an explanatory message. The method checks existence, symmetry, and edge order. It does not check that every valid cover edge exists, that an edge has no intermediate node, that all nodes are reachable, or that learned models match their extents.

## describe

```
def describe(): String
```

Take a node snapshot, sort by descending extent size and ascending ID, format each node with parent/child ID lists, and join the records with newlines. Since `Node.toString` formats the common vector, output cost includes all formatted vector components as well as the  $O(N \log N)$  sort.

## 11. Auxiliary merge and topology maintenance

**Call-path distinction.** The following helpers exist in the class, but `addIntent` does not invoke the global coalescing or cover-rebuild methods. Normal insertion performs bounded local maintenance instead.

### insertOrMergeCandidate

```
private def insertOrMergeCandidate(candidateInstances): Node
```

1. Create and fit a new candidate immediately, consuming a new ID.
2. Linearly scan for the first equivalent existing node.
3. If found, union candidate instances into it; refit once if its extent changes; return it.
4. Otherwise insert and return the candidate.

An ID can be consumed even when the temporary candidate is discarded. Existing edges are not adjusted after a merge changes an extent.

### findEquivalent

```
private def findEquivalent(candidate: Node): Option[Node]
```

Scan `nodeMap` in insertion order and return the first node satisfying learned mutual equivalence. Worst-case cost is a full scan with extent-based classification at every node. If several equivalents exist, linked-map order determines the survivor selected.

### coalesceEquivalentNodes

```
private def coalesceEquivalentNodes(): Unit
```

1. Repeat passes until a pass finds no merge.
2. Snapshot current nodes and compare pairs in index order.
3. When two still-present nodes are equivalent, retain the lower ID and union the dropped node's instances into it, causing a refit.
4. Remove the dropped node from `nodeMap`.
5. Stop that pass after one merge and restart with a fresh snapshot.

Restarting is necessary because refitting the survivor can change equivalence relationships. The routine can be expensive—at least quadratic comparisons per pass and potentially many passes.

**Topology caveat:** it removes the dropped node only from `nodeMap`; it does not redirect or remove IDs in neighbors' parent/child sets. A subsequent `rebuildCoverRelations` is needed to restore clean topology.

## rebuildCoverRelations

```
private def rebuildCoverRelations(): Unit
```

1. Clear every node's parents and children.
2. For every ordered pair of distinct nodes, test whether the first is strictly more general.
3. If so, scan all nodes for a third concept strictly between them.
4. Add the edge only when no intermediate concept exists.

This reconstructs the Hasse cover relation from symbolic extents and restores symmetric adjacency. It is  $O(N^3)$  set-relation tests in the straightforward implementation, so it is suitable as global maintenance rather than the per-insertion path.

## mostSpecificNodeContaining

```
private def mostSpecificNodeContaining(symbol: String): Option[Node]
```

Filter nodes whose symbolic extent contains the requested symbol, sort by ascending extent size then ID, and return the first. "Most specific" therefore means smallest symbolic extent, with deterministic ID tie-breaking. Complexity is  $O(N \log N)$  after symbol membership tests.

# 12. Node construction and input utilities

---

## createNode

```
private def createNode(instances: Iterable[Instance]): Node
```

1. Deduplicate instances by symbol.
2. Atomically fetch and increment `nextId`.
3. Construct a `Node`, whose constructor requires a nonempty extent and fits its initial CVA model.

Creation does not insert the node into `nodeMap`. The caller decides whether a temporary candidate becomes persistent. IDs remain unique even if candidates are discarded; gaps are normal.

## deduplicateInstances

```
private def deduplicateInstances(xs): Vector[Instance]
```

Traverse input in order and write each instance into a `LinkedHashMap` keyed by symbol. A repeated symbol replaces the stored value. Return map values as a vector.

The order is based on the first insertion position of each symbol, while the retained value is the last occurrence assigned to that symbol. Expected time is  $O(K)$ , with  $O(U)$  space for  $U$  unique symbols.

## validateInstance

```
private def validateInstance(instance: Instance): Unit
```

1. Require a nonempty symbol.
2. Require the vector element count to equal `cva.dimension`.

Failure throws via `require`. This helper is currently not called elsewhere in `Lattice.scala`, so it documents intended preconditions more than it enforces the public insertion boundary.

## 13. End-to-end state transition

```
incoming Instance
  ↓
fit/cache singleton target
  ↓
sample closest bottom nodes
  ↓
bounded upward generator search
  ↓
preview accepted ancestors with effectiveSymbols
  ↓
recursively create/reuse local union candidates
  ↓
materialize node and repair nearby symbolic edges
  ↓
add instance to accepted extents and refit their CVA models
  ↓
return the singleton's canonical lattice node
```

## 14. Complexity and behavioral summary

| Concern            | Behavior                                                                                                                                   |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Insertion search   | Bounded by samples, starts, visits, recursion depth, and children per generator; extent scans and CVA operations remain data-dependent.    |
| Consistency        | Models remain unchanged during structural work; accepted models refit afterward, preserving per-call cache validity.                       |
| Topology           | Edges use strict symbolic inclusion. Local insertion does not guarantee the globally complete Hasse cover relation.                        |
| Equivalence        | Mutual learned coverage, optionally strengthened by mutual common-vector classification.                                                   |
| Determinism        | ID tie-breakers and linked insertion order help, but bounded sampling and caller-supplied randomness in diagnostics affect selected nodes. |
| Approximation      | A compatible or equivalent node outside the searched neighborhood can be missed. Learned containment need not satisfy exact FCA laws.      |
| Global maintenance | Full equivalence coalescing and $O(N^3)$ cover rebuilding exist as private helpers but are not part of normal <code>addIntent</code> .     |

**Final interpretation.** `Lattice` is best understood as a bounded, incremental, learned concept graph. The code uses CVA semantics to decide which concepts resemble or classify one another, while using symbolic extents to maintain a true partial-order direction. Its method design emphasizes local updates, stable per-insertion observations, and controlled runtime over exhaustive reconstruction.

Source analyzed: `src/cvafca/Lattice.scala`. Supporting behavioral interpretation uses the called `Node` and `CVA` APIs.